

NSI - Numérique et Sciences Informatiques

Th5Ch5 - Recherche Textuelle

Rappels

Rappelons tout d'abord qu'une chaîne de caractères est un objet de type `str` composé de caractères. Chaque caractère d'une chaîne est repéré par son **index** dans la chaîne. Les index commencent à 0.

Prenons l'exemple de la chaîne `s='abracadabra'`. Alors, `s[0]` a pour valeur le caractère 'a', `s[3]` a pour valeur le même caractère et `s[4]` le caractère 'c'.

La fonction `len` renvoie le nombre de caractères d'une chaîne. Ici, `len(s)` a pour valeur 11.

Problématique

Les algorithmes qui permettent de trouver une sous-chaîne de caractères dans une chaîne de caractères plus grande sont des "grands classiques" de l'algorithmique. On parle aussi de recherche d'un motif (sous-chaîne) dans un texte.

Par exemple, voici le texte suivant :

"Les sanglots longs des violons de l'automne blessent mon cœur d'une langueur monotone. Tout suffoquant et blême, quand sonne l'heure, je me souviens des jours anciens et je pleure."

Question : le motif "vio" est-il présent dans le texte ci-dessus, si oui, en quelle(s) position(s) ? (la numérotation d'une chaîne de caractères commence à zéro et les espaces sont considérés comme des caractères).

Réponse : on trouve le motif "vio" en position 23.

Contexte

La bio-informatique est une discipline où les algorithmes de recherche textuelle sont utilisés.

Bio-informatique

Comme son nom l'indique, la bio-informatique est issue de la rencontre de l'informatique et de la biologie : la récolte des données en biologie a connu une très forte augmentation ces 30 dernières années. Pour analyser cette grande quantité de données de manière efficace, les scientifiques ont de plus en plus recouru au traitement automatique de l'information, c'est-à-dire à l'informatique.

Algorithme "Naïf"

Dans la suite on considère donc deux chaînes de caractères **texte** et **motif**, de longueurs respectives **n** et **p**. Bien entendu, si $p > n$ la recherche de **motif** dans **texte** échoue. En pratique on a $1 \leq p \leq n$, et même p beaucoup plus petit que n . Dans toute la suite on cherche donc la première occurrence d'un **motif** de longueur **p** dans un **texte** de longueur **n**.

Principe de recherche

À un moment donné de la recherche, on observe une **fenêtre** de taille **p** du **texte** complet, sur laquelle on aligne le **motif**, et on regarde si il y a bien correspondance.

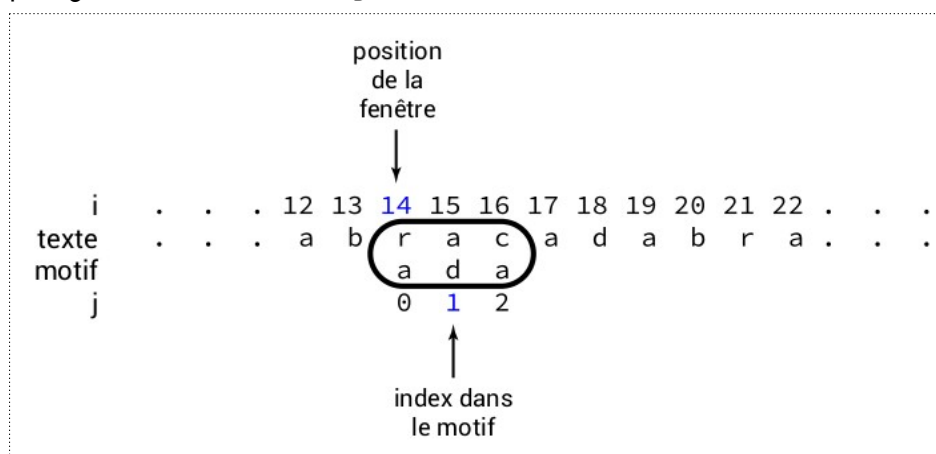
S'il n'y a pas correspondance, on recommencera la recherche avec une fenêtre décalée vers la droite (la fenêtre se déplace de gauche à droite) dans le texte.

Nous noterons **i** la **position** de la fenêtre dans le **texte** : c'est l'**index** du premier caractère du texte qui apparaît dans la fenêtre.

Nous noterons **j** l'index dans le **motif** du **caractère du motif** que nous comparons avec son analogue du texte : on compare **motif[j]** avec **texte[i + j]**.

La recherche peut se faire à condition que $i + p \leq n$ puisque les caractères du texte qui apparaissent dans la fenêtre ont pour index $i, i + 1, \dots, i + p - 1$.

Dans la figure ci-dessous, on représente une fenêtre à la position **i = 14** dans le texte, pour un motif de longueur **p = 3**. On compare le caractère 'd' qui figure à l'index **j = 1** du motif avec le caractère 'a' qui figure à l'index **i + j = 15** dans le texte.



Quand la fenêtre présente un défaut de correspondance entre les caractères du texte et ceux du motif, on déplace la fenêtre. Si le motif correspond parfaitement au texte dévoilé dans la fenêtre, on a trouvé une occurrence à la position **i**.

Comment comparer le motif et le texte ?

L'algorithme naïf consiste simplement à comparer un à un, de gauche à droite, les caractères du texte apparaissant dans la fenêtre avec ceux du motif. En cas de non-correspondance on avance simplement la fenêtre d'une unité vers la droite.

Par exemple, dans la situation suivante,

i	.	.	.	12	13	14	15	16	17	18	19	20	21	22	.	.	.
texte	.	.	.	a	b	r	a	c	a	d	a	b	r	a	.	.	.
motif						a	d	a									
j						0	1	2									

on compare le 'a' du motif avec le 'r' du texte, obtenant immédiatement une différence : on peut avancer la fenêtre en incrémentant i, qui passe de 14 à 15.

Dans la nouvelle fenêtre, le premier caractère coïncide bien :

i	.	.	.	12	13	14	15	16	17	18	19	20	21	22	.	.	.
texte	.	.	.	a	b	r	a	c	a	d	a	b	r	a	.	.	.
motif							a	d	a								
j							0	1	2								

et on incrémente j pour tester les caractères suivants, 'd' et 'c' :

i	.	.	.	12	13	14	15	16	17	18	19	20	21	22	.	.	.
texte	.	.	.	a	b	r	a	c	a	d	a	b	r	a	.	.	.
motif							a	d	a								
j							0	1	2								

On est à nouveau en situation d'échec, et on effectue donc $i = i + 1$ et $j = 0$

A faire vous même 1

Écrire en Python une fonction **correspondance** qui recherche un motif dans un texte, pour une fenêtre en position **i**, puis qui renvoie un couple formé d'un booléen **ok**, égal à **True** si on a trouvé une occurrence du motif et à **False** sinon, et un entier **decalage** qui indique de combien on souhaite augmenter la position **i** de la fenêtre pour la prochaine recherche. Dans le cas où le booléen **ok** vaut **True**, c'est-à-dire si on trouve une occurrence du motif, la valeur de **decalage** n'aura aucune importance.

```
def correspondance(texte, motif, p, i):
    ...
    ...
    return (ok, decalage)
```

Bien entendu vous testerez votre fonction sur un exemple de motif ADN.

A faire vous-même 2

Écrire maintenant la fonction **recherche** qui parcourt un texte pour chercher un motif en utilisant la fonction **correspondance** et qui renvoie la position de la première occurrence du motif ou -1 si celui-ci n'a pas été trouvé.

L'algorithme est le suivant :

```
Fonction Recherche ( texte, motif) :
    Tant que indexTexte + longueurMotif ≤ longueurTexte
        rechercher une correspondance du motif dans le texte à
        la position indexTexte
        si on a trouvé le motif alors
            retourner la position ( indexTexte )
        sinon
            modifier indexTexte en tenant compte du décalage
            indiqué par la fonction correspondance
```

Testez votre fonction sur quelques exemples simples puis sur l'exemple de la page 2.

Cet algorithme naïf peut, selon les situations demander un très grand nombre de comparaisons, ce qui peut entraîner un très long temps de "calcul" avec des chaînes très très longues.

L'algorithme de Boyer-Moore permet de faire mieux en termes de comparaisons à effectuer

Algorithme de Boyer-Moore (Version simplifiée d'après Horspool)

Principes de l'algorithme.

La première idée consiste à comparer le motif avec la portion du texte qui apparaît dans la fenêtre de droite à gauche, et non pas de gauche à droite. Ainsi, on fait décroître j à partir de $p - 1$ jusqu'à trouver que le caractère qui lui fait face dans le texte, c'est-à-dire $x = \text{texte}[i + j]$, est différent du caractère $y = \text{motif}[j]$ du motif.

La deuxième idée consiste à opérer un décalage de la fenêtre qui varie en fonction de la paire de caractères qui ont révélé la non-correspondance, c'est-à-dire en fonction de (x, y) .

Déroulement

Nous considérons ici la recherche du motif '**dab**' dans le texte '**abracadabra**' .

Avec nos notations, $p = 3$, $n = 11$ et la première occurrence du motif dans le texte apparaît en position $i = 6$.

On commence avec la fenêtre tout à gauche, c'est-à-dire avec $i = 0$

```
abracadabra
dab
```

Comme on commence à comparer de droite à gauche, c'est pour $j = 2$ qu'il y a non-correspondance : `motif[2]=='b' != 'r'==texte[0 + 2]` .

On note $x = 'r'$ le caractère du **texte** qui ne correspond pas à $y = 'b'$ le caractère du **motif** qui lui fait face. De combien peut-on décaler la fenêtre ? Comme x n'apparaît nulle part dans le motif, on peut carrément décaler le motif de $p = 3$ unités vers la droite !

```
abracadabra
      dab
```

Ainsi on se retrouve avec $i = 3$ et le premier échec intervient avec $j = 2$, où le caractère $x = \text{texte}[3 + 2]=='a'$ du texte est distinct du caractère face à lui dans le motif, c'est-à-dire $y = \text{motif}[2]=='b'$.

Mais à la différence du cas précédent, le caractère x apparaît bien dans le motif. On déplace donc la fenêtre d'une unité vers la droite.

Voici la prochaine étape :

```
abracadabra
      dab
```

$i = 4$, $x = 'd'$, $y = 'b'$, décalage de 2

Finalement avec $i = 6$, on trouve la première occurrence du motif.

```
abracadabra
      dab
```

À faire vous-même 3

Appliquez l'algorithme précédent au cas suivant :

chaîne :

```
CAATGTCTGCACCAAGACGCCGGCAGGTGCAGACCTTCGTTATAGGCGATGATTTGGAACCTACTAGTGGGTCTCTTAGGCCGAGCGGT
TCCGAGAGATAGTGAAAGATGGCTGGGCTGTGAAGGGAAGGAGTCGTGAAAGCGCGAACACGAGTGTGCGCAAGCGCAGCGCCTTAGTA
TGCTCCAGTGTAGAAGCTCCGGCGTCCCGTCTAACCGTACGCTGTCCCCGGTACATGGAGCTAATAGGCTTTACTGCCCAATATGACCC
CGCGCCGCGACAAAACAATAACAGTTT
```

motif : ACCTTCG

Auteur(s)		David Roche			NSI - Algorithmique	
Rév.	1	Date	12/03/21		Terminale	Page 6/8

Calcul du décalage.

Dans le cas où x n'apparaît pas du tout dans le motif, il convient de déplacer la fenêtre pour qu'elle débute juste à droite du couple (x, y) qui a provoqué l'échec de la recherche. Autrement dit, dans ce cas, le décalage est $\delta = j + 1$.

Dans le cas où x apparaît dans le motif, il convient de déplacer la fenêtre pour que x apparaisse juste au-dessus de la lettre du motif qui lui est égale. Si on note r la position de x la plus à droite dans le motif, il s'agit donc d'utiliser un décalage de $\delta = j - r$ si cette quantité est strictement positive. À défaut, (c'est-à-dire si $\delta \leq 0$) on se contentera d'un décalage d'une unité, comme dans l'algorithme naïf.

Programmation de l'algorithme

Il faut donc commencer par calculer un dictionnaire dont les clés sont les caractères du motif et les valeurs la position la plus à droite du caractère.

C'est ce que réalise la fonction `calculeADroite`.

Dans le cas du mot `maman`, par exemple, on exécute tour à tour des affectations

```
aDroite['m'] = 0
aDroite['a'] = 1
aDroite['m'] = 2
aDroite['a'] = 3
aDroite['n'] = 4
```

de sorte qu'à la fin de l'exécution, `aDroite['m']` est bien égal à 2, position la plus à droite de la lettre 'm' dans le mot 'maman'.

Cela dit, on voudrait calculer le décalage de la fenêtre même quand le caractère qui provoque l'échec n'apparaît pas dans le motif. Mais `aDroite['Z']` par exemple n'existe pas et demander sa valeur déclenche une erreur d'exécution. C'est pourquoi on a écrit la fonction `droite` qui renvoie `-1` si le caractère n'est pas dans le dictionnaire `aDroite`.

```
def calculeADroite(motif, p):
    # remplit (partiellement) un dictionnaire pour donner les positions les plus à
    # droite de chaque caractère
    global aDroite
    aDroite = {}
    for j in range(p):
        aDroite[motif[j]] = j

def droite(c):
    global aDroite
    # renvoie -1 si c n'est pas dans le motif ou sinon aDroite[c]
    if c in aDroite.keys():
        return aDroite[c]
    else:
        return -1
```

On a utilisé une variable globale pour le dictionnaire aDroite . Ce n'est pas toujours une bonne pratique, mais elle semble ici raisonnable.

Nouvelle version de a fonction correspondance

Les deux seules différences dans l'écriture de la fonction correspondance résident dans le parcours du motif de droite à gauche et dans le calcul du décalage, qui n'est pas égal à 1 mais se déduit du dictionnaire aDroite.

```
def correspondance(texte, motif, p, i):
    for j in range(p - 1, -1, -1): # j varie de p-1 à 0 inclus en décroissant
        x = texte[i + j]
        if x != motif[j]:
            decalage = max(1, j - droite(x))
            return (False, decalage)
    return (True, 0)
```

Nouvelle version de la fonction cherche

On modifie très légèrement l'écriture de la fonction cherche en ajoutant, en ligne 4, l'instruction de calcul du dictionnaire utile aux décalages.

```
def cherche(texte, motif):
    n = len(texte)
    p = len(motif)
    calculeADroite(motif, p)
    i = 0
    while i + p <= n:
        ok, decalage = correspondance(texte, motif, p, i)
        if ok:
            return i
        else:
            i = i + decalage
    return -1
```

À faire vous-même 4

Testez le programme précédent avec l'exemple proposé au "À faire vous-même 3" puis avec l'exemple de la page 2.